

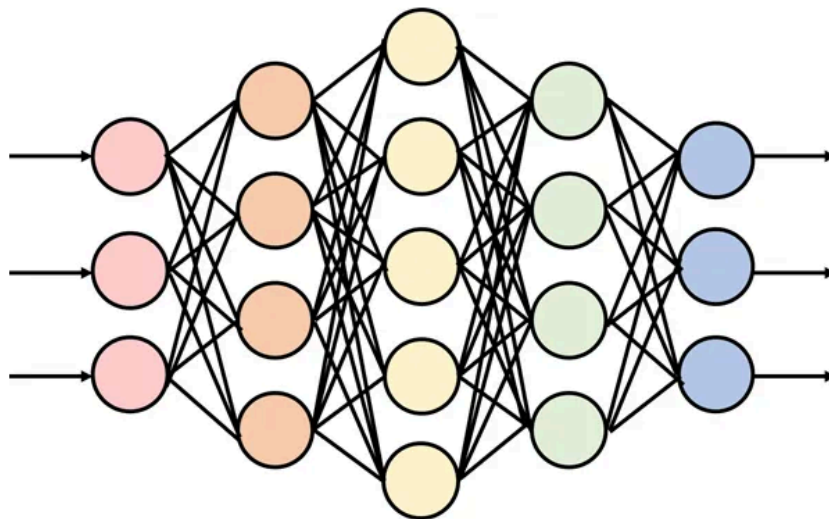
EAI Lab1 Handouts

Outline:

1. How NN Works?
2. Layer Introduction
3. Image Normalization, Data Augmentation, Learning Rate & Update Strategy, Batch Size, Data Cross Validation
4. ResNet18
5. MNIST
6. CIFAR-10
7. Task Requirements
8. Sample Code Introduction

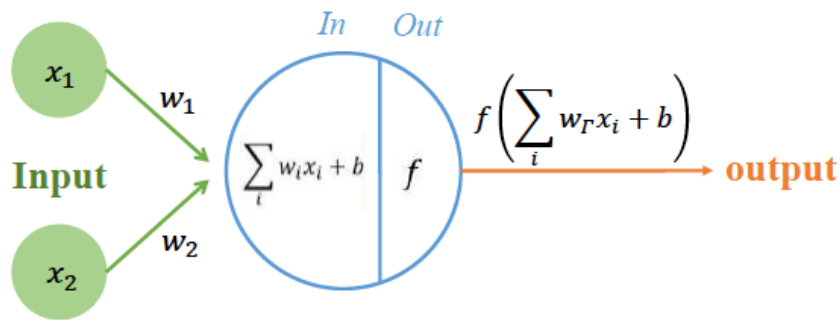
How NN Works?

Neural Network(NN) 神經網路，是一種模仿人類神經網路的運算模式。



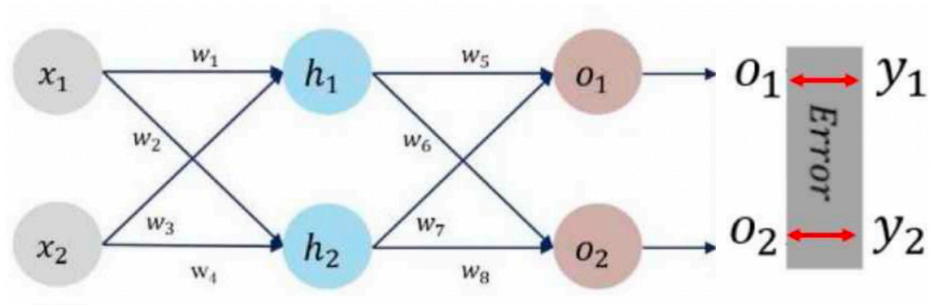
以上圖為例，第一排粉色的神經元層是 Input Layer，負責接收所有資料的輸入，中間第二、三、四排的橘、黃、綠色神經元層是 Hidden Layer，負責處理各式不同形式的權重轉換，最後一排藍色的神經元層是 Output Layer，負責輸出最終結果權重。

神經元的基本運算方式：



以上圖為例， x_1, x_2 作為 Data Input，當 Input 要送到下一層的神經元前，會需要乘上各自的權重 w 並加上偏差 b ，在這邊就會是 $x_1 \times w_1 + b_1, x_2 \times w_2 + b_2$ ，分別完成後會加總起來作為下一層神經元的輸入： $\sum w_i \times x_i + b_i$ ，最後經過下一層神經元的 Function 後即可得到最後的輸出也就是 $f(\sum w_i \times x_i + b_i)$ 。

Forward Propagation:



理解前面神經元運算的概念後，Forward Propagation 就是其運用的實例，資料在經過權重、偏差的運算後，往前傳輸到最後的 Output Layer 得到我們要的輸出。

$$In_{h1} = x_1 \times w_1 + x_2 \times w_3$$

$$Out_{h1} = f(In_{h1}) = h_1$$

$$In_{h2} = x_1 \times w_2 + x_2 \times w_4$$

$$Out_{h2} = f(In_{h2}) = h_2$$

$$In_{o1} = h_1 \times w_5 + h_2 \times w_7$$

$$Out_{o1} = f(In_{o1}) = o_1$$

$$In_{o2} = h_1 \times w_6 + h_2 \times w_8$$

$$Out_{o2} = f(In_{o2}) = o_2$$

① Note

得到最後的輸出 o_1, o_2 後我們就可以拿它來計算誤差，這裡以 MSE 為例可得：

$$Error = \frac{1}{2} \sum_{i=1}^2 (o_i - y_i)^2$$

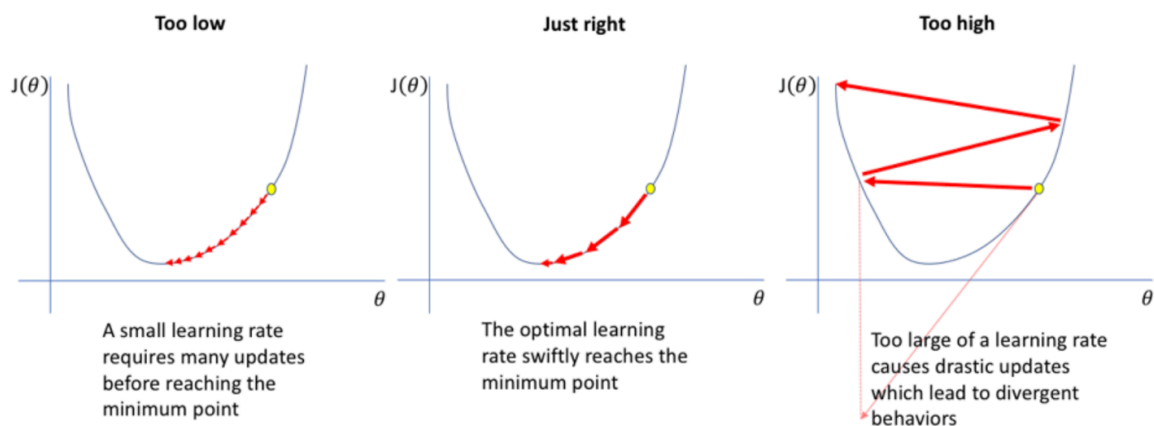
Gradient Descent:

Gradient Descent 是找最佳解的一種方法，用來尋找 Function 的最小值點，在 Neural Network 中，我們使用它來最小化 Loss Function，從而找到最佳的權重 w 和偏差 b 參數。另外，要計算一個函數 $f(x)$ 的 Gradient 有一個前提，就是這個函數要是任意可微分函數。

$$W^{new} = W^{old} - \eta \nabla_w E$$

$$b^{new} = b^{old} - \eta \nabla_b E$$

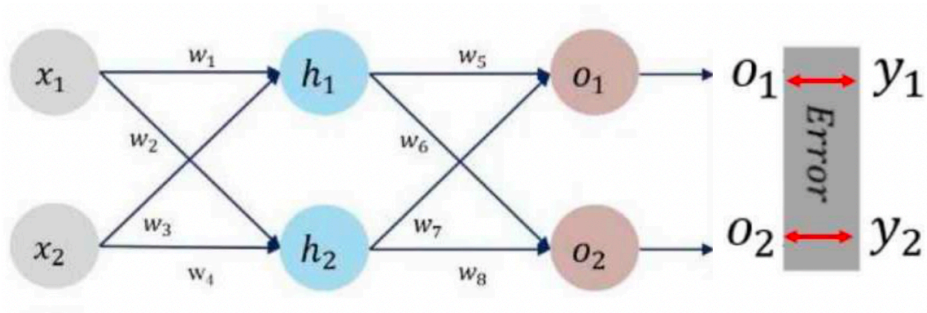
其中 η 是 Learning Rate， $\nabla_w E$ 是 Loss 對權重 w 的 Gradient。



此外，設定一個剛好的 Learning Rate 是加速模型收斂的因素之一，以左圖為例，若 Learning Rate 太小，要到最低點的位置可能會花費較多的 Epoch 才能達成；中間圖顯示恰好的 Learning Rate 可以幫助我們快速、安全地達到最低點；右圖則表示若 Learning Rate 過大，可能會發生震盪的情況，無法達到最低點，甚至是跳出 Local Minimum 的位置。

Backward Propagation:

緊接著我們將要使用 Backward Propagation 來完成模型權重的更新，需要計算 Error 誤差對於欲更新之參數的梯度。



舉例來說，根據上圖我們可以先計算 w_5 到 w_8 的 Gradient(誤差對於權重的變化)，以 w_5 為例：

$$\delta_5 = \frac{\partial Error}{\partial w_5}, \text{ 利用 } chain \text{ rule 可以推導出：}$$

$$\delta_5 = \frac{\partial Error}{\partial w_5} = \frac{\partial Error}{\partial o_1} * \frac{\partial o_1}{\partial In_{o1}} * \frac{\partial In_{o1}}{\partial w_5}$$

$$\because Error = \frac{1}{2} \sum_{i=1}^2 (o_i - y_i)^2$$

$$\therefore \text{對 } o_1 \text{ 偏微分後可得 } \frac{\partial Error}{\partial o_1} = o_1 - y_1$$

$$\because o_1 = Out_{o1} = Sigmoid(In_{o1})$$

$$\therefore \text{對 } In_{o1} \text{ 偏微分後可得 } \frac{\partial o_1}{\partial In_{o1}} = o_1 * (1 - o_1)$$

$$\because In_{o1} = w_5 * h_1 + w_7 * h_2$$

$$\therefore \text{對 } w_5 \text{ 偏微分後可得 } \frac{\partial In_{o1}}{\partial w_5} = h_1$$

最後計算出 δ_5 後，再根據 $w^{new} = w^{old} - \eta * \delta_5$ 來更新權重。

下一步我們就可以再往前推一層，計算 w_1 到 w_4 的 Gradient，以 w_1 為例：

$$\delta_1 = \frac{\partial Error}{\partial w_1} = \frac{\partial Error}{\partial o_1} * \frac{\partial o_1}{\partial w_1} + \frac{\partial Error}{\partial o_2} * \frac{\partial o_2}{\partial w_1}$$

$$= \frac{\partial Error}{\partial o_1} * \frac{\partial o_1}{\partial In_{o1}} * \frac{\partial In_{o1}}{\partial h_1} * \frac{\partial h_1}{\partial In_{h1}} * \frac{\partial In_{h1}}{\partial w_1} + \frac{\partial Error}{\partial o_2} * \frac{\partial o_2}{\partial In_{o2}} * \frac{\partial In_{o2}}{\partial h_1} * \frac{\partial h_1}{\partial In_{h1}} * \frac{\partial In_{h1}}{\partial w_1}$$

$$= \left(\frac{\partial Error}{\partial o_1} * \frac{\partial o_1}{\partial In_{o1}} * \frac{\partial In_{o1}}{\partial h_1} + \frac{\partial Error}{\partial o_2} * \frac{\partial o_2}{\partial In_{o2}} * \frac{\partial In_{o2}}{\partial h_1} \right) * \frac{\partial h_1}{\partial In_{h1}} * \frac{\partial In_{h1}}{\partial w_1}$$

$$= [(o_1 - y_1) \times sigmoid'(In_{o1}) \times w_5 + (o_2 - y_2) \times sigmoid'(In_{o2}) \times w_6] \times sigmoid'(In_{h1}) \times$$

最後計算出 δ_1 後，同樣透過 $w^{new} = w^{old} - \eta * \delta_1$ 來更新權重。

Layer Intro

Inner-Product Layer

Linear Layer 是神經網路中最基本的 Layer 之一，它執行簡單的線性變換，也被稱為 Fully-Connected Layer。

Inner-Product Layer - Forward Propagation

這裡 使用簡單的 $Y = XW + B$ 進行傳遞，其中 X 是 Input Data， Y 是 Output Data， W 是 Weight Data， B 是 Bias Data。

Inner-product layer - Backward propagation

在反向傳遞中，我們需要 計算 Loss Function 對於輸入 X 權重 W 和偏置 b 的梯度。假設我們已經得到了 Loss Function 對輸出的梯度 $\frac{\partial E}{\partial Y}$ ，那麼根據 Chain Rule，我們可以推導出：

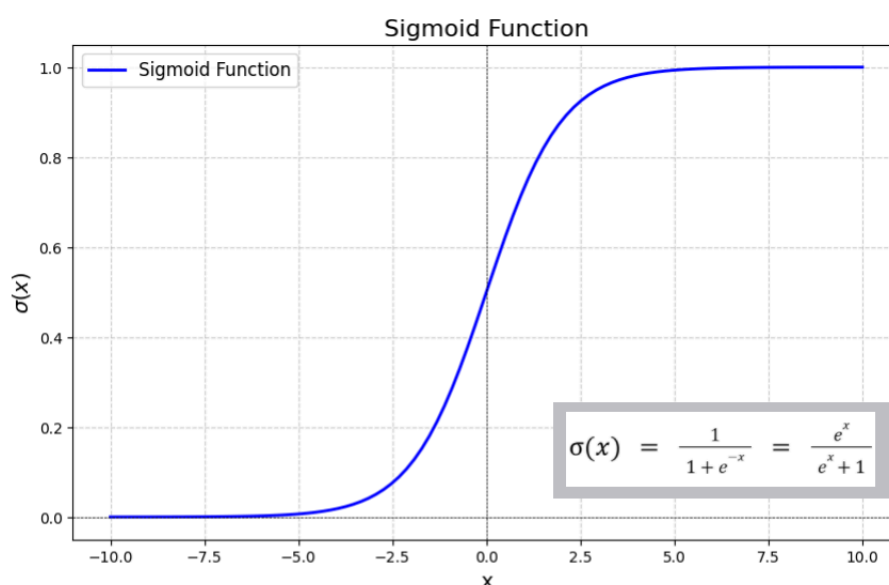
$$\begin{aligned}\frac{\partial E}{\partial X} &= \frac{\partial E}{\partial Y} \frac{\partial Y}{\partial X}, \text{ 又根據 } Y = XW + B, \text{ 可以得到 } \frac{\partial E}{\partial X} = \frac{\partial E}{\partial Y} \frac{\partial Y}{\partial X} = \frac{\partial E}{\partial Y} W \\ \frac{\partial E}{\partial W} &= \frac{\partial E}{\partial Y} \frac{\partial Y}{\partial W}, \text{ 又根據 } Y = XW + B, \text{ 可以得到 } \frac{\partial E}{\partial W} = \frac{\partial E}{\partial Y} \frac{\partial Y}{\partial W} = X \frac{\partial E}{\partial Y} \\ \frac{\partial E}{\partial B} &= \frac{\partial E}{\partial Y} \frac{\partial Y}{\partial B}, \text{ 又根據 } Y = XW + B, \text{ 可以得到 } \frac{\partial E}{\partial B} = \frac{\partial E}{\partial Y} \frac{\partial Y}{\partial B} = \frac{\partial E}{\partial Y} 1\end{aligned}$$

最後再利用這些得到的梯度進行反向傳遞即可。

Activation Layer

Activation Layer 的功能是負責處理神經網路中非線性轉換的部分，透過對神經元的輸入進行非線性轉換，決定神經元是否要輸出訊號，使模型能夠解決更為複雜的問題。如果沒有 Activation Layer，不論神經網路有多少層，整體運算都只是線性變換的疊加，無法處理複雜的非線性問題。

Activation Layer - Sigmoid Function



Sigmoid Function 是神經網路中最常見的 Activation Function 之一，具有特徵性的 S 型曲線，能將任意實數值壓縮到(0,1)區間內，適合表示機率。

Activation layer - Sigmoid Function - Forward Propagation

在向前傳遞時，Sigmoid 層直接對輸入進行 Sigmoid Function 的轉換：

$$y = \sigma(x) = \frac{1}{1 + e^{-x}}$$

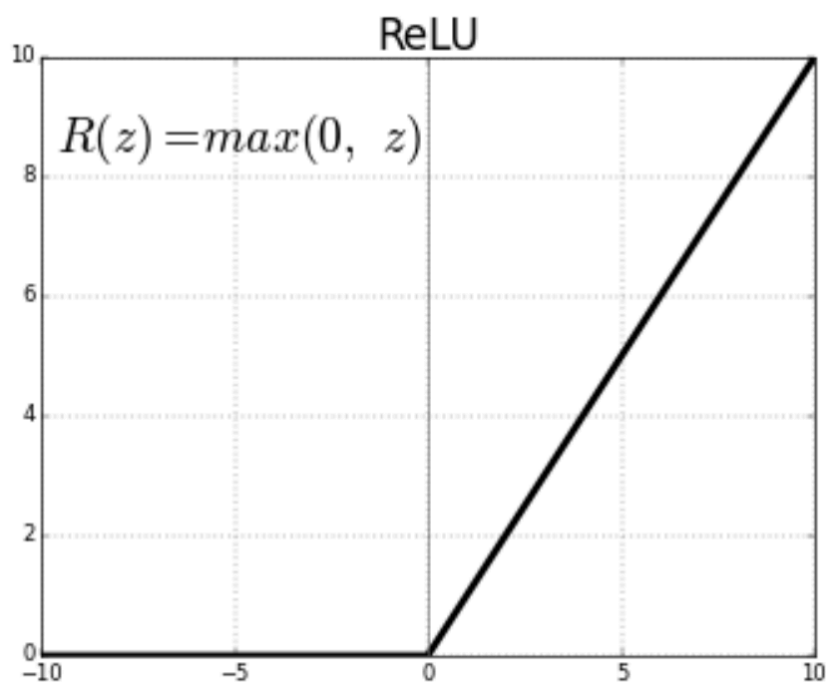
其中 x 代表前一層的輸出， y 代表著 Sigmoid 的輸出。

Activation layer - Sigmoid Function - Backward Propagation

在反向傳遞時，我們需要找到 Loss Function對 x 的梯度，因此根據 Sigmoid 的公式以及 Chain Rule 我們可以推導出：

$$\frac{\partial E}{\partial x} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial E}{\partial y} \frac{d}{dx} f(x)$$

Activation Layer - Rectified Function



ReLU (Rectified Linear Unit) 也是目前深度學習中常用的 Activation Function，因其簡單高效且能有效解決梯度消失問題而廣受歡迎，其數學表達式也很簡單，當輸入大於 0 時直接傳遞，小於 0 時則傳遞 0：

$$y = f(x) = \max(0, x)$$

Activation Layer - Rectified Function - Forward Propagation

在前向傳播過程中，ReLU層對每個輸入元素使用 ReLU Function：

$$y_i = \text{ReLU}(x_i) = \max(0, x_i)$$

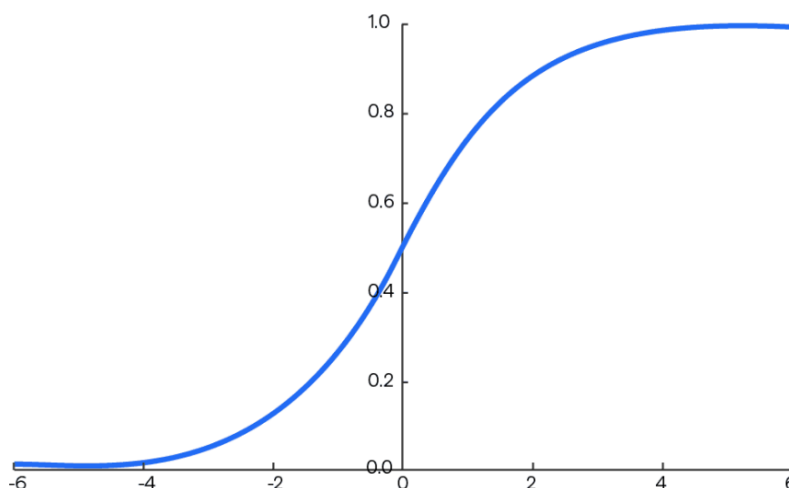
Activation Layer - Rectified Function - Backward Propagation

在反向傳遞時，我們一樣要找 Loss Function對 x 的梯度，同樣根據 ReLU 的公式以及 Chain Rule 我們可以推導出：

$$\frac{\partial E}{\partial x} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial E}{\partial y} \frac{d}{dx} f(x)$$

Activation Layer - Softmax Function

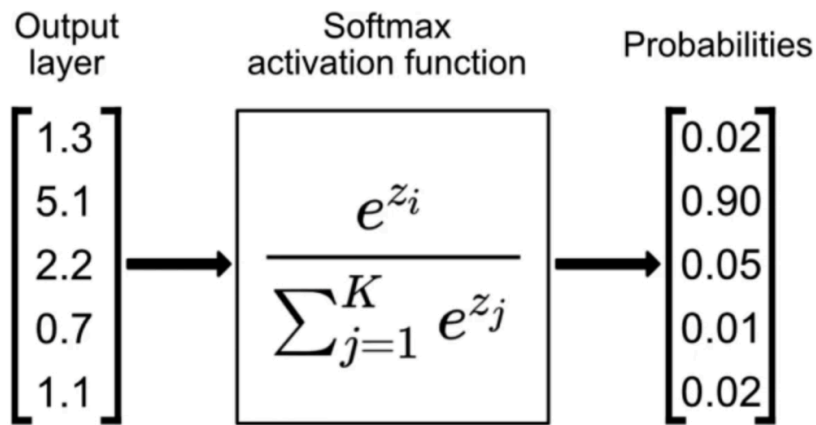
Softmax Function



Softmax 是深度學習中用於多分類問題的 Activation Function，Softmax 能將任意實數值轉換為範圍在 (0,1) 之間的機率值，且所有輸出的總和恆為 1，其數學表達式為：

$$y_i = \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Activation Layer - Softmax Function - Forward Propagation



Softmax 的向前傳遞包含了兩個部分，首先計算指數的總和 $\sum_{j=1}^K e^{z_j}$ ，接著再帶入公式進行歸一化的計算得出 $\text{softmax}(z_i)$ 的輸出。

Activation Layer - Softmax Function - Backward Propagation

Softmax Function 的偏導數為：

當 $i = j$ 時， $\frac{\partial y_i}{\partial z_j} = y_i(1 - y_i)$ ；當 $i \neq j$ 時， $\frac{\partial y_i}{\partial z_j} = -y_i y_j$

假設 Loss Function 對 Softmax 輸出的梯度為 $\frac{\partial E}{\partial y}$ ，那對輸入的梯度就會是

$$\frac{\partial E}{\partial z_i} = \sum_{j=1}^K \frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial z_i}。$$

展開再帶入上述的偏導數後，可以簡化得到：

$$\frac{\partial E}{\partial z_i} = y_i \left(\frac{\partial E}{\partial y_i} - \sum_{j=1}^K \frac{\partial E}{\partial y_j} y_j \right)$$

在實際應用中，若配合著 Cross Entropy 作為 Loss Function，對於 True Label t_i 則可以簡化為

$$\frac{\partial E}{\partial z_i} = y_i - t_i$$

Activation Layer - Others

其他常見的 Activation Function 還包含了 Step, Tanh, Leaky ReLUs, ELUs。

Step Function :

當 $x \geq 0$ 時， $f(x) = 1$ ，當 $x < 0$ 時， $f(x) = 0$ 。是最早期的 Activation Function，模擬神經元的開關特性，但不可微分，無法用於現代深度學習的梯度下降訓練，同時因為輸出只有 0 和 1 兩個值，缺乏細緻的表達能力。

Tanh function :

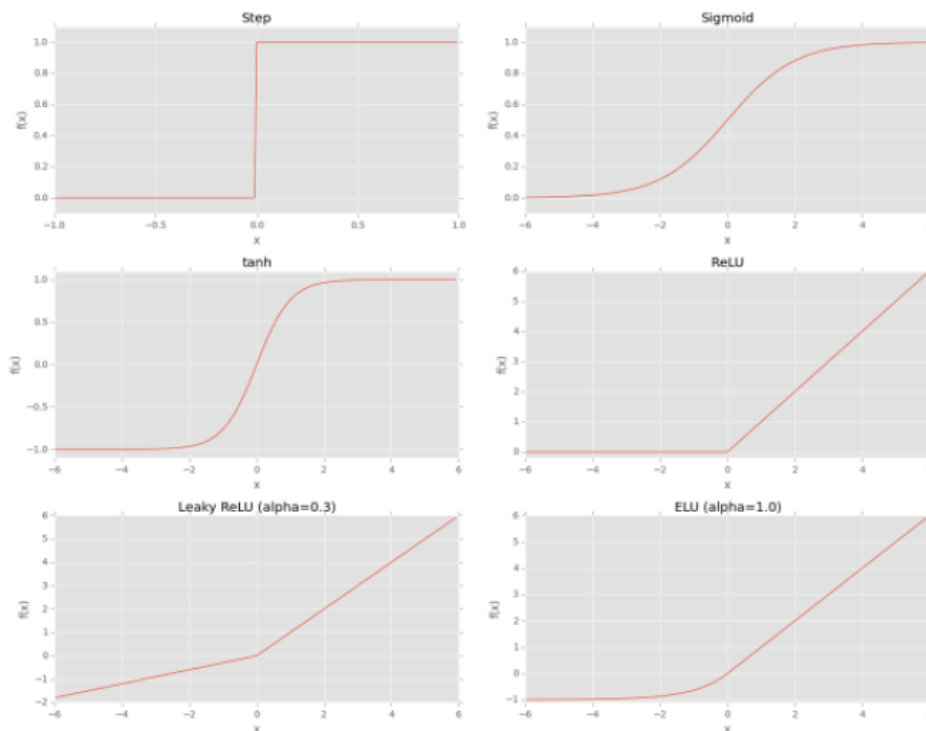
$f(x) = \tanh(x)$ ，輸出範圍在 $[-1, 1]$ 之間，以 0 為中心，有助於訓練穩定性，但仍然存在梯度消失問題，在深層網路中表現不佳。

Leaky ReLU :

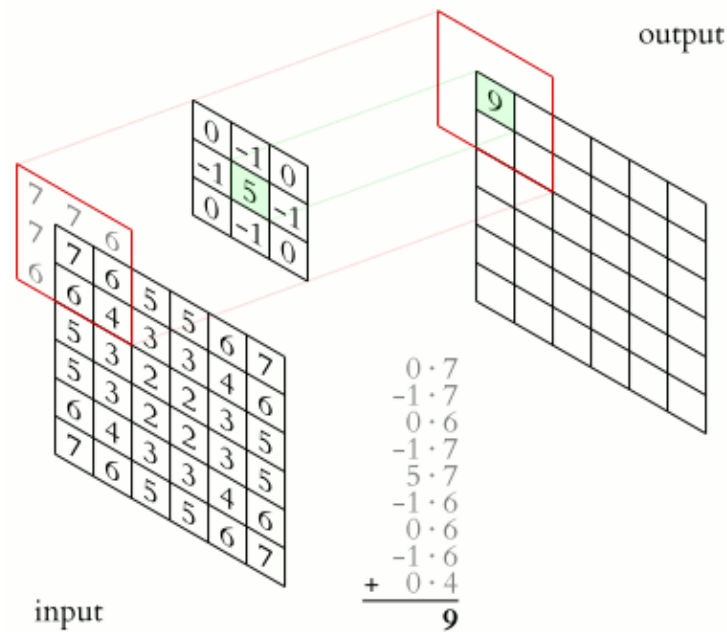
當 $x > 0$ 時， $f(x) = x$ ，當 $x \leq 0$ 時， $f(x) = \alpha x$ ，Leaky ReLU 解決 ReLU 負數死亡問題，當神經元輸入為負時，仍能傳遞微小的梯度信號，其計算簡單，並且保持 ReLU 的大部分優勢。

ELU(Exponential Linear Unit) :

當 $x > 0$ 時， $f(x) = x$ ，當 $x \leq 0$ 時， $f(x) = \alpha(e^x - 1)$ ，ELU 擁有平滑的負數部分，使用指數函數而非線性函數處理負值，同樣解決 ReLU 負數死亡問題。



Convolution Layer



2D Convolution Layer 是使用一個 Kernel 來從輸入圖像中提取特定特徵的操作。Kernel 是一個權重矩陣，與輸入相乘來提取相關特徵，Kernel 的維度決定了 Convolution 的命名（例如 2D Convolution 中，Kernel 是 2D 矩陣）。而 Filter 則是多個 Kernel 的串聯，每個 Kernel 對應輸入的特定 Channel，Filter 的維度比 Kernel 多一維，例如在 2D Convolution 中，Filter 是 3D 矩陣（本質上是 2D 矩陣的串聯），對於 Kernel 維度為 $h \times w$ 、Input Channel 數為 k 的 CNN 層，Filter 維度為 $h \times w \times k$ 。

除了 Input Data, Kernel 外，在 Convolution Layer 的運算中也會加入 Padding 和 Stride 參數，Padding 指的是在圖像邊界添加像素值的數量，而 Stride 則是 Kernel 在圖像上移動的步距。

📌 Important

而輸出的尺寸則可以透過上述參數計算出來：

$$output\ size = \frac{input\ size - kernel\ size + 2 \times padding}{stride}$$

Convolution/Stride = s, only compute convolution on pixels $x_{1+i*s, 1+j*s}$

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

Padding = p = 1, Stride = s = 2

X: m x n

W: k x k

$$\rightarrow Y: \left(\frac{m-k+2p}{s} + 1 \right) \times \left(\frac{n-k+2p}{s} + 1 \right)$$

e.g.

X: m x n, m = n = 5

W: k x k, k = 3

$$\left(\frac{5-3+2*1}{2} + 1 \right) \times \left(\frac{5-3+2*1}{2} + 1 \right) = (3,3)$$

① Note

那為什麼我們需要使用 CNN 而不用 NN 就好呢？有幾個原因：

1. 減少參數數量：透過權重共享來減少需要訓練的參數
2. 加速執行時間：較少的參數可以讓運算的速度更快
3. 結合圖像特徵：能夠有效提取局部特徵並組合成更複雜的特徵
4. 自動特徵提取：不再需要手動設計卷積核

💡 Tip

參數計算 Example，以 LeNet 為例：

$input\ size = 28 \times 28 \times 1$, $chaannels = 6$, $kernel = 5 \times 5 \times 1$, $stride = 1$, $padding = 0$

$$output\ size = \left(\frac{28 - 5 + 2 \times 0}{1} \right) + 1 = 24$$

$number\ of\ \frac{neurons}{unit}\ within\ conv\ layer = output\ height \times output\ width \times number\ of\ fe$

$$maps(channels) = 24 \times 24 \times 6 = 3456\ units$$

$number\ of\ training\ parameters\ or\ weights\ within\ conv\ layer(without\ weight\ sharing)$

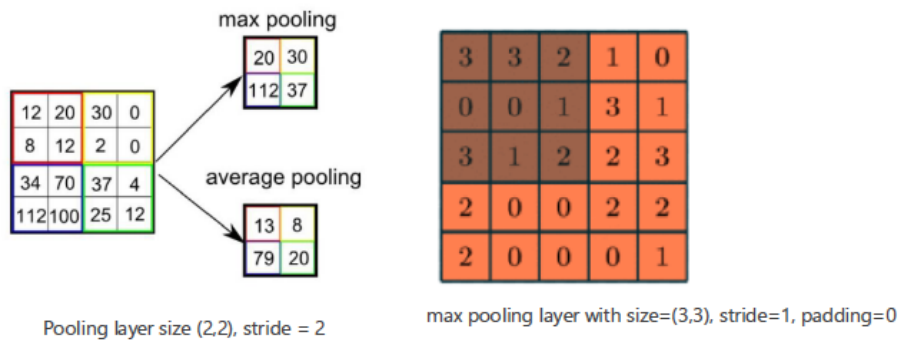
$$= 3456 \times ((5 \times 5 \times 1) + 1\ bias) = 89856\ weights$$

$number\ of\ training\ parameters\ or\ weights\ with\ weight\ sharing(with\ weight\ sharing)$

$$= 6 \times ((5 \times 5 \times 1) + 1\ bias) = 156\ weights$$

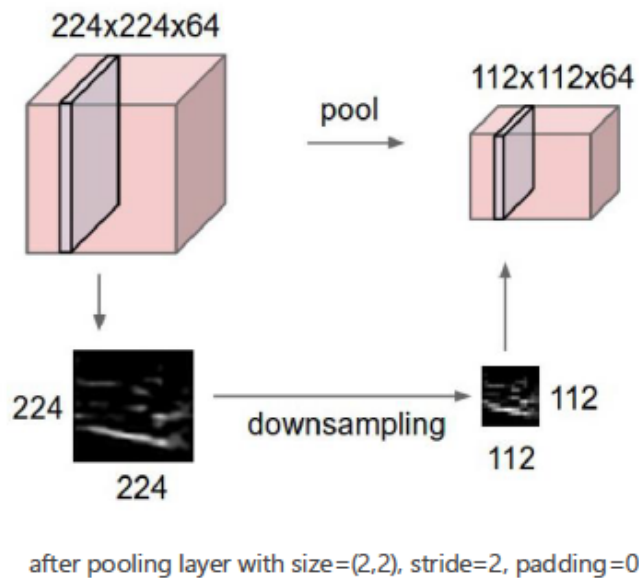
Pooling Layer

Pooling Layer 是卷積神經網路中的重要功能，透過滑動窗口方法來減少模型維度，能夠有效降低特徵圖的空間尺寸，同時保留重要的特徵資訊。



這邊會介紹 Max Pooling 以及 Average Pooling：

1. Max Pooling 是在每個池化窗口中選取最大值，可以丟棄不必要或雜訊特徵，保留最顯著的特徵。
2. Average Pooling 是計算池化窗口內所有值的平均值，可以提供更平滑的特徵表示，相比 Max Pooling，能保留更多的背景資訊。



根據上圖可以看到 Pool Size (2,2), Stride = 2，經過 Pooling 後將輸入特徵圖的尺寸減半。

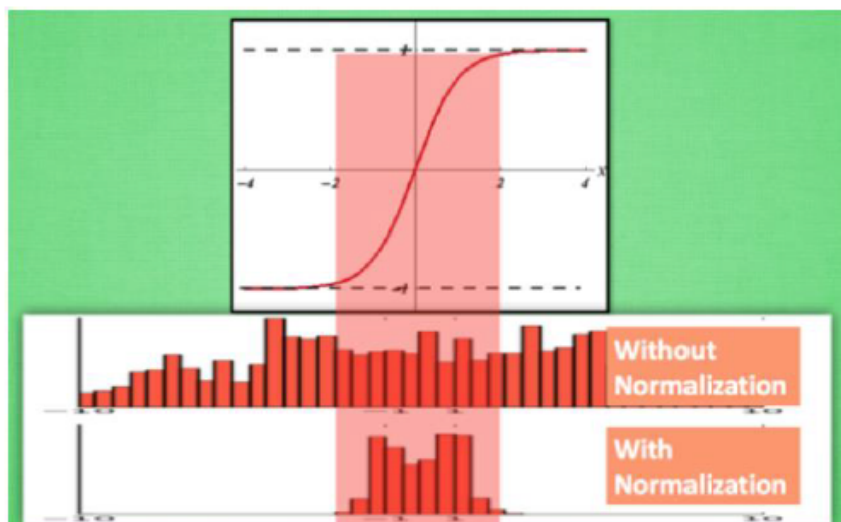
① Note

Pooling 主要有幾個優點：

1. 他可以顯著減少特徵圖的空間尺寸，降低後續層的計算複雜度。
2. 防止過擬合，通過降維減少模型複雜度，提高模型的泛化能力。
3. 增強計算效率，減少記憶體使用量，加速網路的前向和反向傳播。

Batch Normalization Layer

Batch Normalization Layer 是深度學習中的重要技術，他的核心作用是讓不同特徵具有相同的縮放尺度，在沒有正規化的情況下，不同特徵可能有完全不同的數值範圍，這會造成訓練困難。此外，他可以加速訓練過程，能夠用更大的學習率進行更快的訓練，並避免避免協變量偏移問題使得網路訓練更加穩定和高效。同時，Batch Normalization對於避免梯度消失特別有效，尤其是在使用 Sigmoid 和 Tanh 等傳統 Activation Function 時，這些 Function 在深層網路中容易產生梯度消失，而 Batch Normalization 能有效改善這個問題。

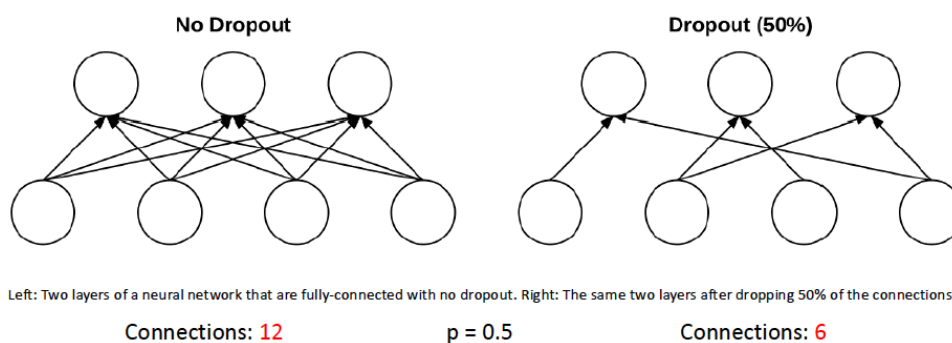


① Note

從上圖可以看到 Batch Normalization 的效果對比，在使用正規化後，所有特徵被縮放到相似的範圍，損失函數的等高線變得更加圓形，更容易優化。

Dropout Layer

Dropout Layer 主要功能是透過隨機丟棄多個冗餘節點來泛化模型，防止 Overfitting 問題的發生，讓 Model 學會在部分資訊缺失的情況下仍能做出正確判斷，增強面對雜訊的抗干擾能力。



① Note

從上圖可以看到，當沒有 Dropout 時，兩個全連接層之間有完整的 12 個連接，所有神經元都參與計算，使用 Dropout 時 ($p = 0.5$)，隨機丟棄 50% 的連接，只剩下 6 個連接，被丟棄的神經元在當前訓練步驟中不參與前向傳播和反向傳播 (p 表示丟棄的機率)。

Flatten Layer

Flatten Layer 的核心作用是進行維度重塑，將多維輸入轉換成一維形式。在 CNN 架構中，Flatten Layer 扮演橋接角色，通常用於從 Convolution Layer 到 FC Layer 的過渡階段。

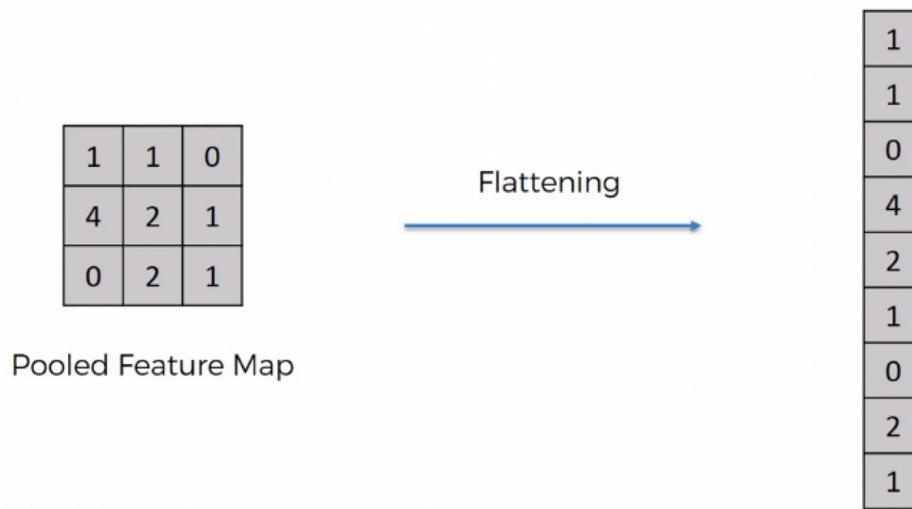


Image Normalization, Data Augmentation, Learning Rate & Update Strategy, Batch Size, Data Cross Validation

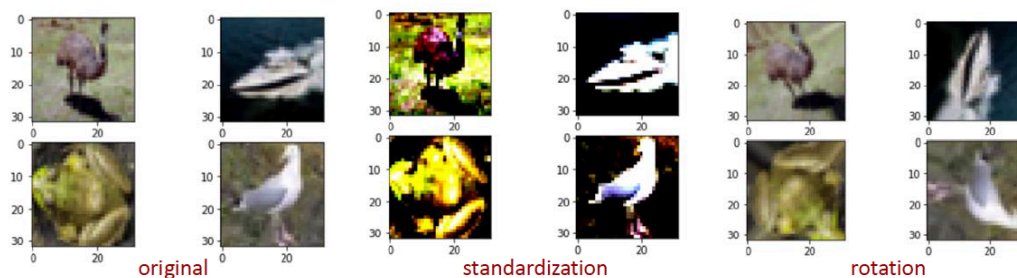
Image Normalization

Image Normalization 又稱為圖像正規化，目的是將輸入的像素值進行縮放，常見的有兩種方式：

1. Min/Max Normalization，可以將像素值縮放到 0 ~ 1 或 -1 ~ 1 之間，通常在 Input Range 已知的情況下使用，數學公式： $output = input / 255$ ，在 PyTorch 中也可以透過 `transforms.ToTensor()` 將 `Range(0, 255)` 轉換為 `Range(0.0, 1.0)`。
2. Standardization，這是更常用且效果更好的正規化方法，他的核心概念是將採樣資料集的 Mean 轉換成接近 0，std 轉換成接近 1，以此減少偏差，避免被某部分資料支配，通常用於 Input Range 未知的情況，採用採樣的方式取得統計值，數學公式： $(image - train\ mean) / train\ std$ 。其優勢在於 Mean 接近 0，Unbiased 的資料更有利於 Model 學習，std 接近 1，可以減緩梯度消失和梯度爆炸問題。

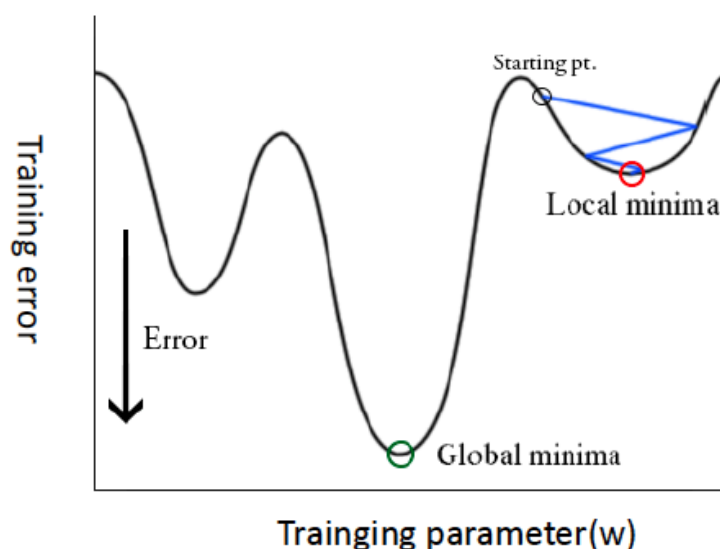
Data Augmentaion

Data Augmentation 是透過對原始圖像進行各種變換來人工增加訓練資料多樣性的方法，可以讓模型學習到更多樣化的特徵，提高泛化能力，常見的變換方法包含：旋轉、調整大小和比例尺寸、改變亮度色溫、翻轉（如水平翻轉）、加入高斯雜訊（Gaussian Noise）等等。



Learning Rate & Update Strategy

在訓練模型時，若採用固定的 Learning Rate，容易陷入 Local Minima 而非 Global Minima。



為此，我們可以延伸出其他調整 Learning Rate 的方式，例如 Learning Rate Decay 和 Cyclical Learning Rates。Learning Rate Decay 指的是 Model 在訓練一定 Epoch 後，對學習率進行衰減，可以讓模型收斂得更好，初期用較大學習率快速逼近最優解，後期用較小學習率精細調整。而 Cyclical Learning Rates 則可以設定學習率的上下限，讓學習率在一個範圍內週期性地衰減或增加，優點在於在遇到 Saddle Point 時不會卡住，週期性變化也可以幫助模型跳出局部最優，尋找更好的解。

Batch Size

$$\theta^* = \arg \min_{\theta} L$$

➤ (Randomly) Pick initial values θ^0

➤ Compute gradient $g^0 = \nabla L^1(\theta^0)$

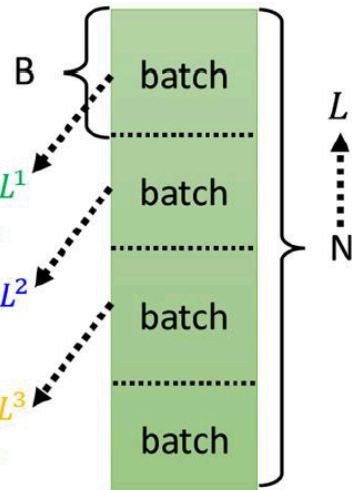
$$\text{update } \theta^1 \leftarrow \theta^0 - \eta g^0$$

➤ Compute gradient $g^1 = \nabla L^2(\theta^1)$

$$\text{update } \theta^2 \leftarrow \theta^1 - \eta g^1$$

➤ Compute gradient $g^2 = \nabla L^3(\theta^2)$

$$\text{update } \theta^3 \leftarrow \theta^2 - \eta g^2$$



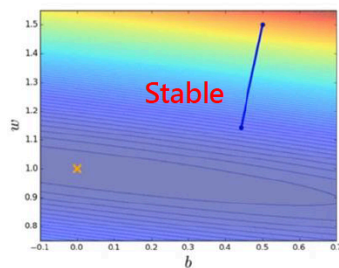
1 epoch = see all the batches once → Shuffle after each epoch

Batch 指的是一次訓練迭代中同時處理的樣本數量。在模型訓練時，我們不一定是一次處理所有資料，許多時候是將資料分成多個 Batch 來進行訓練，而不同 Batch Size 的大小會影響模型訓練的效果。

Consider 20 example(N=20)

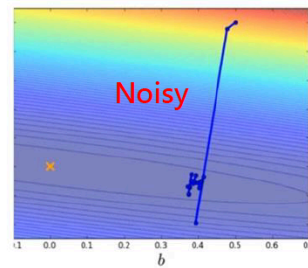
Batch size = N(Full batch)

Update after seeing
all 20 examples



Batch size = 1

Update for each example
Update 20 times in an epoch



① Note

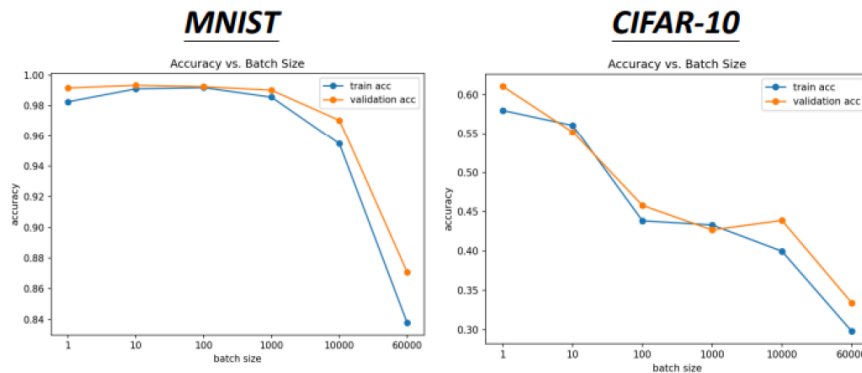
Batch Size 小 :

1. 梯度較 Noisy 但優化效果更好
2. 泛化能力更強
3. 單次更新速度較快 (無平行運算時)
4. 每個 Epoch 時間較長

① Note

Batch Size 大：

1. 梯度較 Stable
2. 單次更新時間較長（無平行運算時）
3. 每個 Epoch 時間較短
4. 優化和泛化效果較差



➤ Smaller batch size has better performance

① Note

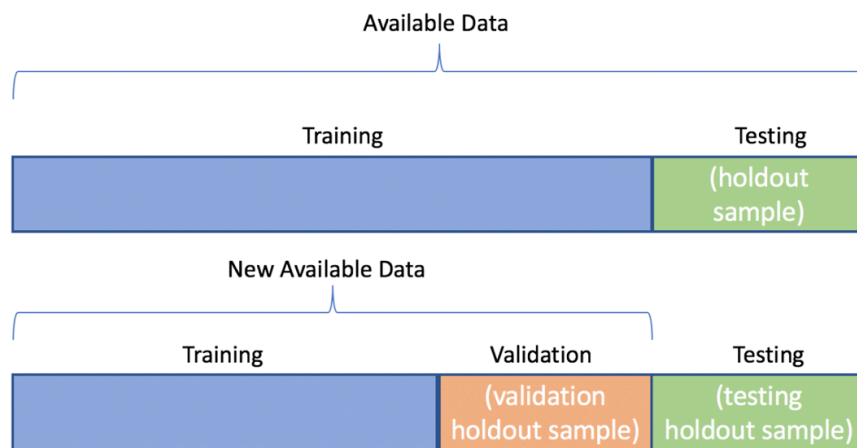
在 MNIST 和 CIFAR10 的訓練中，小的 Batch Size 基本上都能達到更好的訓練效果。

Data Cross Validation

Cross Validation(交叉驗證)在機器學習中是一件很重要的事，他可以評估模型的泛化能力，避免 Overfitting 的發生。常見的有 Training/Validation Split, K-Fold Cross Validation 等等。

1. Training/Validation Split :

這是將原先的 Training Set 切分成 Training Set 和 Validation Set 的方法，這樣的好處在於我們在訓練 Model 的過程中，可以透過觀察 Training Set 和 Validation Set 各自的 Loss/Accuracy 來知道模型是否開始 Overfitting，也更能尚未使用 Test Set 來驗證的情況下，客觀的觀察模型的訓練狀況。



💡 Tip

常見情況下會將 Validation Set 的 Size 設定為 Training Set 的 0.1 ~ 0.2 左右。

2. K-Fold Cross Validation

在 K-fold 方法中，會將整個 Training Set 分割成 K 等份，K 值可以自由調整，每次訓練時，會從這 K 等份中選擇其中 K-1 份作為訓練資料，剩下 1 份作為驗證集。他的優勢在於可以確保原始數據集中的每個觀察值都有機會出現在訓練集和測試集中，相比其他方法通常會產生偏差較小的模型，也不會過度依賴於特定的數據劃分方式，可以透過不同分組訓練結果的平均來減少方差。

📌 Note

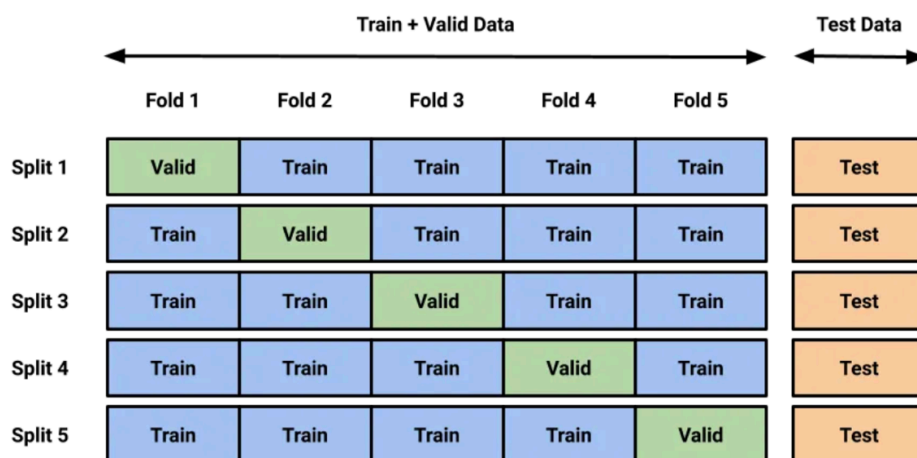
以 5-Fold Cross Validation 為例，假設有 100 筆資料分成 5 份：

Fold 1：用第 2,3,4,5 份訓練，第 1 份驗證

Fold 2：用第 1,3,4,5 份訓練，第 2 份驗證

依此類推，直到所有折疊都當過驗證集

最後取 5 次驗證結果的平均作為模型整體表現



ResNet18

ResNet18是ResNet (Residual Network) 系列中的18層深度殘差網絡，由微軟研究院在2015年提出，主要是為了解決深度神經網絡中的兩個核心問題：

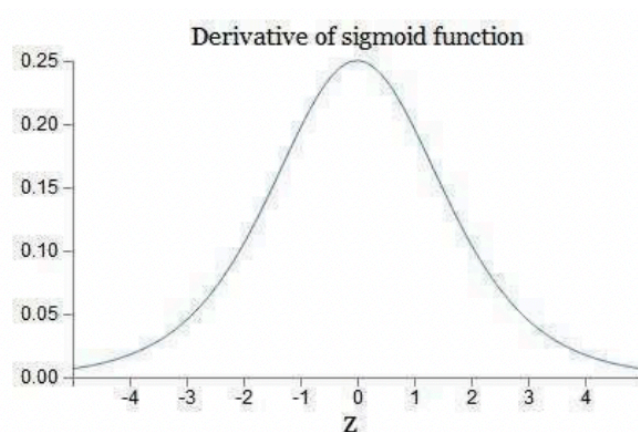
1. 梯度消失問題 (Vanishing Gradient Problem)

在神經網絡的優化過程中，不可避免的都會使用到 Backward Propagation，而當 Model 層數過深時，在 Backward Propagation 過程中 Gradient 會逐層衰減，導致淺層網絡幾乎無法得到有效更新。

$$\begin{aligned}\frac{\partial E_{total}}{\partial w_1} &= \frac{\partial E_{total}}{\partial y_4} \frac{\partial y_4}{\partial z_4} \frac{\partial z_4}{\partial x_4} \frac{\partial x_4}{\partial z_3} \frac{\partial z_3}{\partial x_3} \frac{\partial x_3}{\partial z_2} \frac{\partial z_2}{\partial x_2} \frac{\partial x_2}{\partial z_1} \frac{\partial z_1}{\partial w_1} \\ &= \frac{\partial E_{total}}{\partial y_4} \sigma'(z_4) w_4 \sigma'(z_3) w_3 \sigma'(z_2) w_2 \sigma'(z_1) x_1\end{aligned}$$

① Note

由於初始化的權重通常會是在 0 附近的小數，因此在深層網路中的 Gradient 經過多層的相乘後，亦會趨近於 0，導致權重無法得到更新。



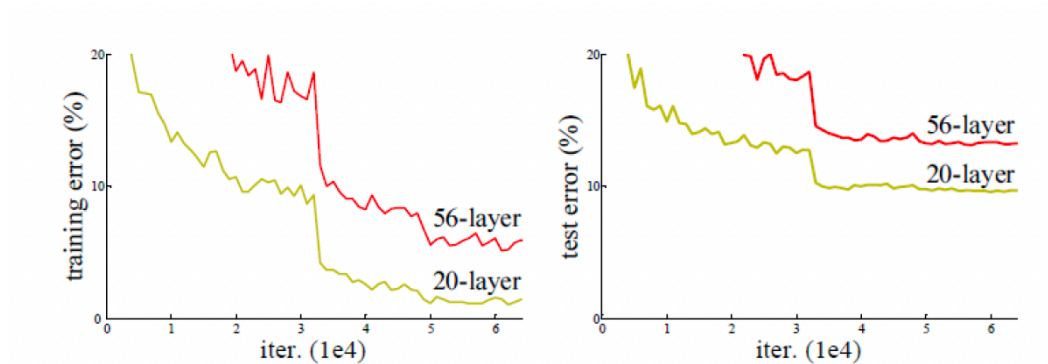
特別是使用 Sigmoid Function 時，其導數範圍在(0, 0.25)之間，多層相乘後梯度變得極小，最後導致無法訓練。

① Note

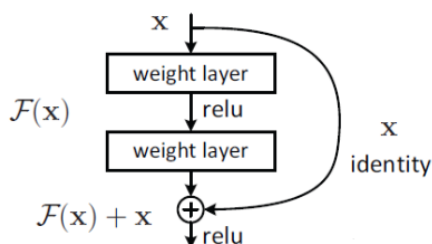
可以使用 Batch Normalization 或是改用 ReLU 來解決問題。

2. 退化問題 (Degradation Problem)

此外，當 Model 深度增加時，訓練誤差反而會增加，這並不是 Overfitting 或 Vanishing Gradient Problem 造成的，而是 Degradation Problem。以下圖為例，56層 Model 的訓練誤差居然比20層 Model 還要高，這表明深層特徵丟失了淺層特徵的原始信息。



為了解決以上問題，ResNet 提出了殘差連接 (Skip Connection) 的概念：



在傳統機器學習的過程中，網路設計大概是：

$$\text{Input } x \rightarrow \text{Mapping Function } F(x) \rightarrow \text{Output } x$$

是在學習找到一個 $F(x)$ 讓我們的輸入可以滿足我們想要的輸出。

但 ResNet 把網路設計成：

$$\text{Input } x \rightarrow \text{Mapping Function } F(x) \rightarrow \text{Output } H(x) = F(x) + x$$

轉換為 學習一個殘差函數 $F(x) = H(x) - x$ ，這樣的好處在於，如果恆等映射是最佳解，網絡只需要學習讓 $F(x) = 0$ ，而且即使 $F(x)$ 學習不好，至少還有原始輸入 x 的信息，梯度也可以直接通過 skip connection 回流，避免梯度消失。

ResNet18 的架構同學可以參考下面這張圖：

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

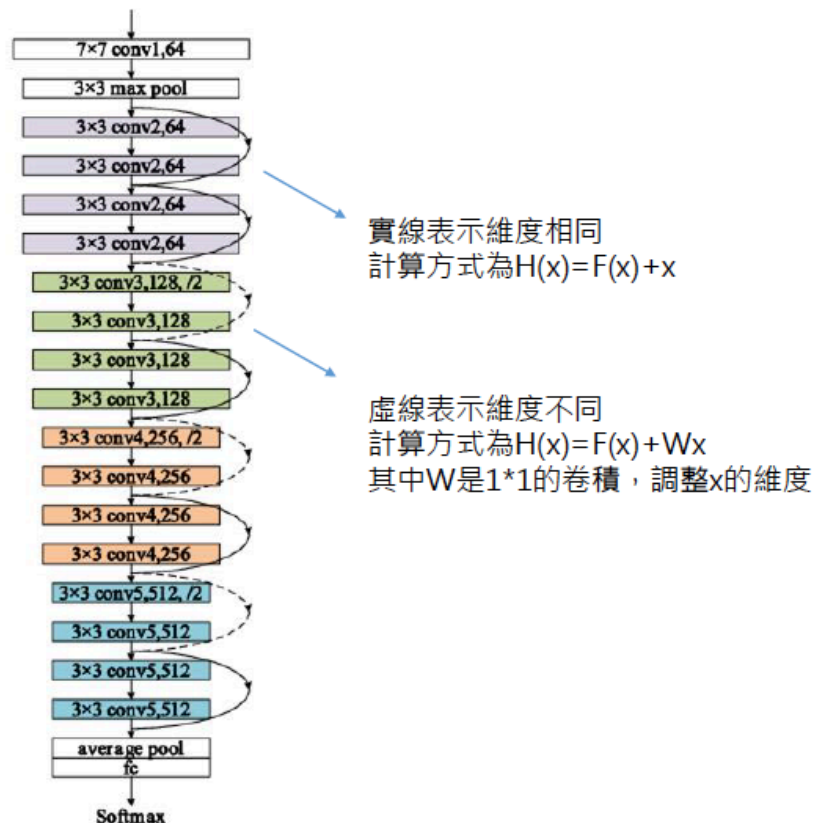
在 ResNet 中使用了兩種 Residual Block：

1. Identity Block

- 輸入和輸出維度相同
- 殘差連接為： $H(x) = F(x) + x$
- 在圖中以實線表示

2. Convolution Block

- 輸入和輸出維度不同，需要通過 1×1 卷積調整維度： $H(x) = F(x) + Wx$
- 在圖中以虛線表示



💡 Tip

同學在作業實作過程中，建議可以先完成先完成 make layer 的 Function 再接上 4 層 Layer。

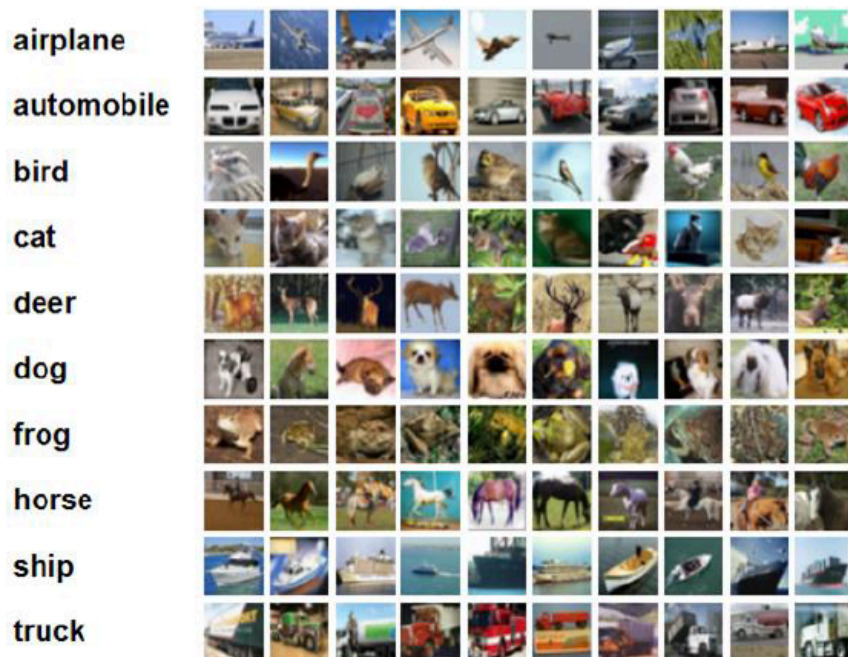
MNIST

MNIST (Modified National Institute of Standards and Technology database) 是機器學習領域中最著名的手寫數字資料集，專門用於機器學習和深度學習的數字識別任務。MNIST 資料集包含兩個主要部分：訓練資料有 60,000 張灰階影像（ 28×28 像素）、測試資料有 10,000 張灰階影像（ 28×28 像素），每張影像皆對應一個標籤。原始資料是以 ubyte 格式儲存，但為了方便處理，資料通常會被轉換為 CSV 格式，在模型的訓練中，也常會將標籤轉換為 One-Hot 編碼格式，以便顯示每個標籤的預測機率。



CIFAR10

CIFAR-10 (Canadian Institute for Advanced Research-10) 是由加拿大高等研究院開發的標準影像分類資料集，是機器學習和深度學習領域中最廣泛使用的基準資料集之一，其中包含了 60,000 張影像，每張影像的尺寸為 32×32 像素，為 RGB 的 3 Channel 輸入，總共有 10 個類別(飛機、汽車、鳥、貓、鹿、狗、青蛙、馬、船、卡車)，每類 6000 張圖片。



Task Requirements

Recommend Training Platform : Colab With Python

Upload Platform : : NCKU Moodle

Upload Compressed "StudentID_lab1.zip" File Including:

- StudentID_lab1-1.ipynb (請注意檔案必須包含程式碼與執行結果)
- StudentID_lab1-2.ipynb (請注意檔案必須包含程式碼與執行結果)
- StudentID_lab1.pdf (此為本次作業 Report)

繳交格式如下:

```
StudentID_lab1 -----> compressed as StudentID_lab1.zip
├── StudentID_lab1-1.ipynb
├── StudentID_lab1-2.ipynb
└── StudentID_lab1.pdf
```

Task 1 (30%)

1. 10% 完成 NN Layer 的實作(包含 Forward & Backward Propagation) :
 - 完成 Inner-Product Layer (Linear Layer)
 - 完成 Activation Layer (ReLU 和 Sigmoid)
 - 完成 Softmax Layer
 - 可加入其他自己會用到的 Layer(但禁止使用 Convolution Layer)
2. 5% 完成 MNIST 訓練設定 :
 - 設定自己的訓練參數
 - 必須包含 Cross-Validation (可使用 Train/Valid 切分或是 K-Fold-Cross-Validation)
3. 10% 設計自己的 NN，並完成 MNIST 的訓練 :
 - 利用先前完成的 Layer 來設計自己的模型，至少需要包含一層 Hidden Layer
 - 正確率需大於 90%
4. 5% Plot 出 Training / Validation Loss & Accuracy 圖。

Task 2 (45%)

1. 10% 設計 ResNet18 Model :
 - 請根據 ResNet18 的架構，並利用 Pytorch 刻出模型
2. 5% 印出 Model 的 Summary (包含 Parameters)
3. 5% 完成 CIFAR10 訓練設定 :
 - 設定自己的訓練參數
4. 10% 利用 ResNet18 完成 CIFAR10 的訓練
 - 正確率需大於 85%

5. 10% 請加入 Image Normalization, Data Augmentation, Different Learning Rate & Update Strategy，並在 Report 中比較差異。
6. 5% Plot 出 Training / Validation Loss & Accuracy 圖。

Task 3 (25%)

Report :

1. 25% 請根據上述你的 Task1.1, Task1.2 的內容，完成書面報告，內容包括但不限於：
 - 說明你的程式實作方式
 - 呈現各項要求的輸出內容（包含 Print 出的數值以及圖片等等）
 - 說明你在完成作業過程中遇到的問題以及你的解決方法

Sample code intro

Task 1 Code

```
1 | from google.colab import drive
2 | drive.mount('/content/drive')
```

```
1 | %cd /content/drive/MyDrive/AI_Lab1
```

在檔案中有附上 MNIST 的 csv 檔，請各位同學先傳至自己的雲端後，我們可以透過這上面的程式碼連結到自己的雲端空間，接著再利用 `%cd /content/drive/Mydrive/檔案位置` 來移動至你在雲端中放檔案的資料夾。

```
1 | class Parameter:
2 |     def __init__(self, data: np.ndarray) -> None:
3 |         self.data = data
4 |         self.grad = None
5 |
6 |
7 | class Module:
8 |     def __call__(self, *args, **kwargs) -> np.ndarray:
9 |         return self.forward(*args, **kwargs)
10 |
11 |     def __repr__(self) -> str:
12 |         layers = '\n'.join([f'  ({k}): {v}' for k, v in self.__dict__.items()])
13 |         return f'{self.__class__.__name__}(\n{layers}\n)'
```

這邊我們建立了兩個 Class : Parameter, Module，Parameter 用來儲存 layer 中的參數及其梯度，Module 則是用來定義神經網路中的 operator。

```

1 class Linear(Module):
2     def __init__(self, in_features, out_features) -> None:
3         super().__init__()
4         self.in_features = in_features
5         self.out_features = out_features
6
7         # Initialize weights and biases
8         init_factor = 0.01
9         self.W = Parameter(np.random.randn(in_features, out_features) * init_
10        self.b = Parameter(np.zeros((1, out_features)))
11
12        # Cache for backward pass
13        self.x = None
14
15    def forward(self, x):
16        # 學生實作部分：return output of linear layer
17
18
19    def backward(self, dy):
20        # 學生實作部分：return gradient w.r.t. input and compute gradients for v
21
22
23    def parameters(self):
24        return self.W, self.b
25
26    def __repr__(self) -> str:
27        return f'{self.__class__.__name__}(in_features={self.in_features}, ou

```

接著就是各個 Layer 的建立，包含了 Linear, ReLU, Sigmoid, Softmax，其中同學要完成 4 個 Class 的 def forward 和 def backward 的程式碼。

① Note

其中 Softmax 的 Backward Propagation 部分，由於我們設定同學們會在後面皆上 Cross Entropy 作為 Loss Function，因此根據講義先前的推導，我們可以使用簡化過後的 Back Propagation，這邊助教已經把簡化部分寫在 CE 的程式碼中，因此同學可以不用寫，但若同學想以其他函數作為 Loss Function 進行訓練，就請將 Softmax 的 Backward Propagation 部分重新編寫。

💡 Tip

除此之外，若同學想要使用其他除 Convolution 的 Layer，也可以在此按照這 4 個 Layer 當作模板自行定義。

```

1 class MLP(Module):
2     def __init__(self) -> None:
3         # 學生實作部分: design your Model architecture here
4
5     def forward(self, x):
6         # 學生實作部分: compute forward pass through your model
7
8     def backward(self, dy):
9         # 學生實作部分: compute backward pass through your model
10
11     def parameters(self):
12         return self.fc1.parameters() + self.fc2.parameters()

```

在建立好 Layer 的 Function 後，我們要來建立自己的 Model，首先要在 `def __init__` 的部分完成自己的 Model 架構，接著完成 `def forward` 和 `def backward` 參數傳遞的部分。

```

1 class MNIST:
2     # root請根據你的檔案位置更改
3     def __init__(self, root='./data', train=True, transform: Callable = None):
4         path = os.path.join(root, 'mnist_train.csv' if train else 'mnist_test')
5         self.data = np.loadtxt(path, delimiter=',')
6         self.transform = transform
7         self.image_size = 28
8         self.num_classes = 10
9         self.classes = np.arange(self.num_classes)
10
11     def __len__(self):
12         return len(self.data)
13
14     def __getitem__(self, idx):
15         label = (self.data[idx, 0] == self.classes).astype(
16             np.float32
17         ) # one-hot encoding
18         image = (
19             self.data[idx, 1:]
20             .reshape(self.image_size * self.image_size)
21             .astype(np.float32)
22         )
23         if self.transform:
24             image = self.transform(image)
25         return image, label

```

接著我們將 MNIST 的資料加入。

❗ Note

同學要記得將 `def __init__` 中的 `root` 位置改成你存放 `mnist_train.csv` 的位置。

```

1  # Separate train_imgs, train_labels into training and validation
2  # root請根據你的檔案位置更改
3  def load_mnist_data(
4      root="./data", batch_size=1, split_ratio=0.1, transform=None
5  ) -> tuple[DataLoader, DataLoader, DataLoader]:
6      def _split_dataset(dataset, split_ratio):
7          # 學生實作部分: split dataset into training and validation sets
8          # hint: return Subset(dataset, train_indices), Subset(dataset, valid_
9
10
11
12
13      trainset = MNIST(root=root, train=True, transform=transform)
14      testset = MNIST(root=root, train=False, transform=transform)
15      trainset, validset = _split_dataset(trainset, split_ratio=split_ratio)
16      trainldr = DataLoader(trainset, batch_size=batch_size)
17      validldr = DataLoader(validset, batch_size=batch_size)
18      testldr = DataLoader(testset, batch_size=batch_size)
19      return trainldr, validldr, testldr

```

接著我們要將 Training Set 的資料切分成 Training Set / Validaiton Set，同學需要完成這邊 `def _split_dataset` 的 Function，完成後會 `return Subset(dataset, train_indices), Subset(dataset, valid_indices)`。

```

1 class CrossEntropyLoss(Module):
2     def __init__(self, epsilon=1e-15) -> None:
3         super().__init__()
4         self.epsilon = epsilon # small value to avoid log(0)
5         self.y_pred = None
6         self.y_true = None
7
8     def forward(self, y_pred, y_true):
9         self.y_pred = np.clip(y_pred, self.epsilon, 1 - self.epsilon)
10        self.y_true = y_true
11        batch_size = y_true.shape[0]
12        loss = -np.sum(y_true * np.log(self.y_pred)) / batch_size
13        return loss
14
15    def backward(self):
16        batch_size = self.y_true.shape[0]
17        grad = (self.y_pred - self.y_true) / batch_size
18        return grad
19
20    def __repr__(self) -> str:
21        return f"{self.__class__.__name__}()"
22
23
24 class SGD:
25     def __init__(self, params: Iterable, lr: float = 1e-3) -> None:
26         self.params = params
27         self.lr = lr
28
29     def step(self):
30         for param in self.params:
31             if param.grad is not None:
32                 param.data -= self.lr * param.grad
33
34     def zero_grad(self):
35         for param in self.params:
36             if param.grad is not None:
37                 param.grad.fill(0)

```

這邊我們提供了 CE 作為 Loss Function，SGD 作為 Optimizer。

📌 Note

同學也可以根據自己的需求替換不同的 Loss Function 及 Optimizer。

```

1  def train_one_epoch(model: Module, trainldr: Iterable, criterion, optimizer)
2      total = 0
3      correct = 0
4      total_loss = 0
5      for x, y in tqdm(trainldr):
6          # 學生實作部分: complete the training process through one epoch
7
8          # Hint:
9          # 1. forward propagation
10         # 2. compute loss
11         # 3. compute accuracy
12         # 4. backward propagation
13         # 5. update parameters
14
15         avg_loss = total_loss / total
16         accuracy = correct / total
17         return avg_loss, accuracy
18
19
20  def evaluate(model: Module, testldr: Iterable) -> tuple[float, float]:
21      total = 0
22      correct = 0
23      total_loss = 0
24      criterion = CrossEntropyLoss() # separate criterion from training
25      for x, y in tqdm(testldr):
26          # 學生實作部分: complete the evaluation process
27          # Hint:
28          # 1. forward propagation
29          # 2. compute total loss
30          # 3. compute correct and total
31
32         avg_loss = total_loss / total
33         accuracy = correct / total
34         return avg_loss, accuracy

```

下一步同學要完成模型訓練的過程，包含了 `def train_one_epoch` 以及 `def evaluate`，且都要記錄並回傳他們的 Loss 和 Accuracy。

③ Note

`def train_one_epoch` 和 `def evaluate` 看似相仿，但他們負責不同的功能，`def train_one_epoch` 負責 Model 的訓練過程，吃入的參數是 Training Set，`def evaluate` 則負責的是 Validation Set 和 Test Set 驗證的部分。

```

1 def transform(x):
2     """map pixels information from range(0, 255) to range(0.01, 1)"""
3     return np.asarray(x) * 0.99 + 0.01
4
5 # "../data"請根據你的檔案位置更改
6 trainldr, validldr, testldr = load_mnist_data(
7     "../data", batch_size=1, transform=transform
8 )
9 print(f"train set: {len(trainldr)} images")
10 print(f"validation set: {len(validldr)} images")
11 print(f"test set: {len(testldr)} images")
12 for x, y in trainldr:
13     print(f"x shape: {x.shape}")
14     print(f"y shape: {y.shape}")
15     break

```

這邊我們要對訓練的資料集進行 Transform，讓參數的範圍從 (0, 255) 轉移至 (0.01, 1)。

① Note

同學記得將 ../data 改成你檔案所放置的位置。

```

1 net = MLP()
2 print(net)
3 # 你可以調整 epochs 和 lr 來觀察不同的訓練效果
4 train_loss, train_acc, valid_loss, valid_acc = train(
5     net, trainldr, validldr, epochs=, lr=
6 )

```

這邊我們將 net 定義為先前建立的 Model，並輸出 Model 的架構，接著就開始訓練。

① Note

同學需要自行設定 Epochs 和 Learning Rate，來找到最適合的設定。

```

1 # Using test_images and test_labels to do the final test
2 test_loss, test_acc = evaluate(net, testldr)
3 print(f"test_loss = {test_loss}, test_acc = {test_acc}")

1 # Plot training and validation loss and accuracy curves
2
3 plt.show()

```

最後將 Test Set 導入訓練完的 net 進行 Evaluate 後，Plot 出 Loss & Accuracy 的圖就完成了本次的 Task 1。

Task 2 Code


```

1 | # Check if CUDA is available
2 | print(torch.cuda.is_available())

```

在 Import 完需要的 Package 後，我們先透過上面的程式碼檢查 CUDA 是否可以使用。

```

1 | # Calculate mean and std of dataset
2 | def get_mean_std(dataset, ratio=1):
3 |     # Get mean and std by sample ratio
4 |     dataloader = torch.utils.data.DataLoader(dataset, batch_size=int(len(data
5 |
6 |     data = next(iter(dataloader))[0]      # get the first iteration data
7 |     mean = np.mean(data.numpy(), axis=(0,2,3))
8 |     std = np.std(data.numpy(), axis=(0,2,3))
9 |     return mean, std
10 |
11 | train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True, downl
12 | test_dataset = torchvision.datasets.CIFAR10(root='./data', train=False, downl
13 |
14 | train_mean, train_std = get_mean_std(train_dataset)
15 | test_mean, test_std = train_mean, train_std
16 | print(train_mean, train_std)
17 | print(test_mean, test_std)

```

下一步我們計算 CIFAR10 這個資料集的平均值和標準差，這兩個參數會用在 Data Augmentation 的 Normalization 中。

```

1 | ##### data augmentation & normalization #####
2 | transform_train = transforms.Compose([
3 |     # 同學實作部分：Add data augmentation here
4 |
5 |
6 |     transforms.ToTensor(), # Transform to tensor
7 |     transforms.Normalize(mean=train_mean, std=train_std), # Normalization
8 | ])
9 |
10 | transform_test = transforms.Compose([
11 |     transforms.ToTensor(),
12 |     transforms.Normalize(mean=test_mean, std=test_std),
13 | ])

```

緊接著就是 Data Augmentation 的部分，同學在這邊請嘗試加入不同的 Data Augmentation 來幫助 Model 的訓練。

```

1 # Dataset
2 trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=T
3 test_ds = torchvision.datasets.CIFAR10(root='./data', train=False, download=T
4
5 image, label = trainset[0]
6 print("image shape: ", image.shape)
7
8 # Split validation dataset
9 torch.manual_seed(43) # Ensure reproducibility
10 val_size = 5000 # Take 5000 images as validation set
11 train_size = len(trainset) - val_size
12 train_ds, val_ds = random_split(trainset, [train_size, val_size])
13 print("train length: ", len(train_ds))
14 print("val length: ", len(val_ds))
15 print("test length: ", len(test_ds))
16
17 # Declare batch size
18 # 學生實作部分: Set batch size
19 BATCH_SIZE =
20 trainloader = torch.utils.data.DataLoader(train_ds, batch_size=BATCH_SIZE, sh
21 valloader = torch.utils.data.DataLoader(val_ds, batch_size=BATCH_SIZE, shuffl
22 testloader = torch.utils.data.DataLoader(test_ds, batch_size=BATCH_SIZE, shuf

```

在處理完 Data 後，我們將其導入進來，這邊請同學自己設定 Batch Size，嘗試找到最適合模型訓練的尺寸。

```

1 class BasicBlock(nn.Module):
2     def __init__(self, in_channels, out_channels, stride=1):
3         super(BasicBlock, self).__init__()
4         # 學生實作部分: Define the two convolutional layers and the shortcut con
5
6
7     def forward(self, x):
8         # 學生實作部分: Define the forward pass using convolutional layers and i
9
10         return out

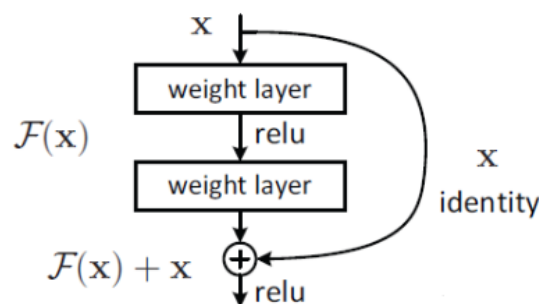
```

在完成資料的前處理後，我們要開始建立 ResNet 的 Model，這邊會分為兩個 Class：BasicBlock, ResNet18。

首先是 BasicBlock，這邊同學需要實作出 def __init__，def forward 兩個 Function。

在 def __init__ 中完成 Layer 的實作，基本上就會是完成下面這張 Shortcut Connection 的架構。

然後在 def forward 中完成參數的傳遞。



```

1 class ResNet18(nn.Module):
2     def __init__(self, num_classes=1000):
3         super(ResNet18, self).__init__()
4         # 學生實作部分: Define the ResNet-18 architecture using BasicBlock
5
6
7     def _make_layer(self, block, out_channels, num_blocks, stride):
8         # 學生實作部分: Define make_layer function to create layers of blocks
9
10        return nn.Sequential(*layers)
11
12    def forward(self, x):
13        # 學生實作部分: Define the forward pass of ResNet-18
14
15        return out

```

下一步就是 ResNet18 的建立，這邊總共有 3 個 Function 要完成：def __init__，def _make_layer，def forward。

在先前講義中 ResNet18 的架構圖可以發現，它具有 4 個大的 Conv Layer，中間包含了小的 Block，def _make_layer 的功能就是在我們傳入前面寫好的 BasicBlock 以及想要的 Output 尺寸大小後，可以幫我們建立起這些 Conv Layer。

def __init__ 則是編寫整個 ResNet18 Model 主要架構的 Function，我們會在其中利用剛寫好的 _make_layer Function 來幫助我們建立 Model。

最後在 def forward 中完成參數的傳遞。

```

1 # Create SummaryWriter
2 writer = SummaryWriter("../tensorboard")
3
4 model = ResNet18(num_classes=10)
5 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
6 model = model.to(device)
7 # Print model summary
8 summary(model, (3, 224, 224))
9
10 # Calculate FLOPs and Params
11 dummy_input = torch.randn(1, 3, 32, 32).to(device)
12 flops, params = profile(model, inputs=(dummy_input, ))
13 print(f"FLOPs: {flops/1e6:.2f} MFLOPs")
14 print(f"Params: {params/1e6:.2f} M")

```

在建立好 Model 架構後，我們一樣先印出他的 Summary 和 FLOPS & Params，來觀察模型每層的形狀以及參數量。

```

1 # Setting parameter
2 EPOCH =
3 pre_epoch =
4 lr =
5
6 # 同學可以根據自己需求調整optimizer, criterion與lr_scheduler
7 criterion = nn.CrossEntropyLoss()
8 optimizer = optim.Adam(model.parameters(), lr=lr)

```

下一步同學需要設定訓練的參數，包含 Epoch, pre_epoch, lr等等，同學可以嘗試不同的設定，也可以更換 Criterion 和 Optimizer。

```
1 best_model_path = 'best_resnet18.pth'
2
3 # Record training and validation metrics
4 train_losses = []
5 train_accuracies = []
6 val_losses = []
7 val_accuracies = []
8 best_val_acc = 0.0
9
10 # Training and validation loop
11 for epoch in range(pre_epoch, EPOCH):
12     model.train()
13     running_loss = 0.0
14     correct = 0
15     total = 0
16     # 學生實作部分: Complete the training and validation loop
17
18
19
20
21     # 儲存最佳模型
22     if val_acc > best_val_acc:
23         best_val_acc = val_acc
24         torch.save(model.state_dict(), best_model_path)
25
26
27     print(f"Epoch [{epoch+1}/{EPOCH}] "
28           f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}% | "
29           f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.2f}%")
```

接著就是訓練模型的部分，同學需要完成 Model 在 Training and Validating 的部分。

📌 Note

這邊同學要記得紀錄 Train/Val 的 Loss/Accuracy，以便後續的畫圖。

💡 Tip

此外，這邊有個儲存最佳模型的程式碼，在 Colab 中，若程式成功運行，就會在左邊檔案夾的部分儲存一個叫 best_resnet18.pth 的檔案，同學可以在訓練完後下載到電腦中，以便下一次若要接續訓練就不用全部重新重跑一次。

```

1  # Load the best model and evaluate on the test set
2  model.eval()
3
4  test_correct = 0
5  test_total = 0
6  with torch.no_grad():
7      for test_inputs, test_labels in testloader:
8          test_inputs, test_labels = test_inputs.to(device), test_labels.to(dev
9          test_outputs = model(test_inputs)
10         _, test_predicted = test_outputs.max(1)
11         test_total += test_labels.size(0)
12         test_correct += test_predicted.eq(test_labels).sum().item()
13 test_accuracy = 100. * test_correct / test_total
14 print(f"Best Model Test Accuracy: {test_accuracy:.2f}%")

```

當 Model 都訓練完成後，我們再利用 Test Set 來檢查模型訓練的成果，並打印出正確率。

```

1  ##### Plot loss & accuracy graph #####
2  import matplotlib.pyplot as plt
3  # 學生實作部分: Plot training and validation loss and accuracy curves
4
5
6  plt.show()

```

最後，同學需要 Plot 出 Loss & Accuracy 的圖 就完成了本次的 Task 2。